

# Loan Approval Prediction

## Machine Learning Project

```
In [ ]: %pip install pandas numpy matplotlib seaborn scikit-learn imbalanced-learn
```

```
Requirement already satisfied: pandas in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (2.3.3)
Requirement already satisfied: numpy in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (2.4.3)
Requirement already satisfied: matplotlib in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (3.10.8)
Requirement already satisfied: seaborn in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (0.13.2)
Requirement already satisfied: scikit-learn in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (1.8.0)
Collecting imbalanced-learn
  Downloading imbalanced_learn-0.14.1-py3-none-any.whl.metadata (8.9 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from pandas) (2026.1.post1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.10.0 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from scikit-learn) (3.6.0)
Collecting sklearn-compat<0.2,>=0.1.5 (from imbalanced-learn)
  Downloading sklearn_compat-0.1.5-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: six>=1.5 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading imbalanced_learn-0.14.1-py3-none-any.whl (235 kB)
Downloading sklearn_compat-0.1.5-py3-none-any.whl (20 kB)
Installing collected packages: sklearn-compat, imbalanced-learn
```

[illegible]

```

----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 2/2 [imbalanced-learn]

```

Successfully installed imbalanced-learn-0.14.1 sklearn-compat-0.1.5

[notice] A new release of pip is available: 25.3 -> 26.1.1

[notice] To update, run: C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

In [ ]:

```

In [3]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    RocCurveDisplay
)

from imblearn.over_sampling import SMOTE

```

In [5]: df = pd.read\_csv('data/loan\_prediction.csv')

In [6]: df.head()

Out[6]:

|   | Loan_ID  | Gender | Married | Dependents | Education    | Self_Employed | ApplicantIncome | CoapplicantIncome |
|---|----------|--------|---------|------------|--------------|---------------|-----------------|-------------------|
| 0 | LP001002 | Male   | No      | 0          | Graduate     | No            | 5849            | 0                 |
| 1 | LP001003 | Male   | Yes     | 1          | Graduate     | No            | 4583            | 0                 |
| 2 | LP001005 | Male   | Yes     | 0          | Graduate     | Yes           | 3000            | 0                 |
| 3 | LP001006 | Male   | Yes     | 0          | Not Graduate | No            | 2583            | 0                 |
| 4 | LP001008 | Male   | No      | 0          | Graduate     | No            | 6000            | 0                 |

In [7]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
Index: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                614 non-null   object
1   Gender                 601 non-null   object
2   Married                611 non-null   object
3   Dependents             599 non-null   object
4   Education              614 non-null   object
5   Self_Employed          582 non-null   object
6   ApplicantIncome        614 non-null   int64
7   CoapplicantIncome      614 non-null   float64
8   LoanAmount             592 non-null   float64
9   Loan_Amount_Term       600 non-null   float64
10  Credit_History          564 non-null   float64
11  Property_Area          614 non-null   object
12  Loan_Status            614 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 67.2+ KB
```

In [8]: `df.isnull().sum()`

```
Out[8]: Loan_ID      0
Gender      13
Married      3
Dependents  15
Education    0
Self_Employed  32
ApplicantIncome    0
CoapplicantIncome  0
LoanAmount      22
Loan_Amount_Term  14
Credit_History   50
Property_Area     0
Loan_Status      0
dtype: int64
```

In [9]: `df.describe()`

Out[9]:

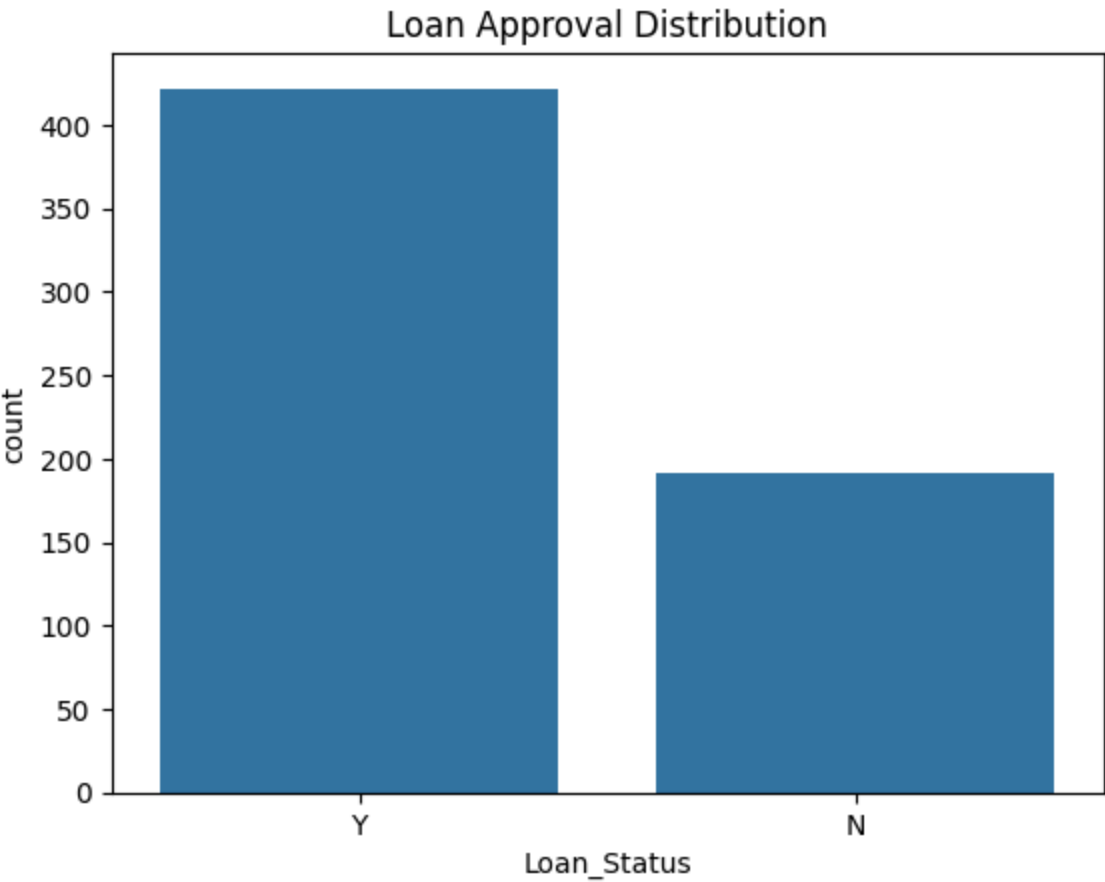
|       | ApplicantIncome | CoapplicantIncome | LoanAmount | Loan_Amount_Term | Credit_Hist |
|-------|-----------------|-------------------|------------|------------------|-------------|
| count | 614.000000      | 614.000000        | 592.000000 | 600.00000        | 564.0000    |
| mean  | 5403.459283     | 1621.245798       | 146.412162 | 342.00000        | 0.8421      |
| std   | 6109.041673     | 2926.248369       | 85.587325  | 65.12041         | 0.3648      |
| min   | 150.000000      | 0.000000          | 9.000000   | 12.00000         | 0.0000      |
| 25%   | 2877.500000     | 0.000000          | 100.000000 | 360.00000        | 1.0000      |
| 50%   | 3812.500000     | 1188.500000       | 128.000000 | 360.00000        | 1.0000      |
| 75%   | 5795.000000     | 2297.250000       | 168.000000 | 360.00000        | 1.0000      |
| max   | 81000.000000    | 41667.000000      | 700.000000 | 480.00000        | 1.0000      |

In [10]:

```
sns.countplot(x='Loan_Status', data=df)

plt.title("Loan Approval Distribution")

plt.show()
```

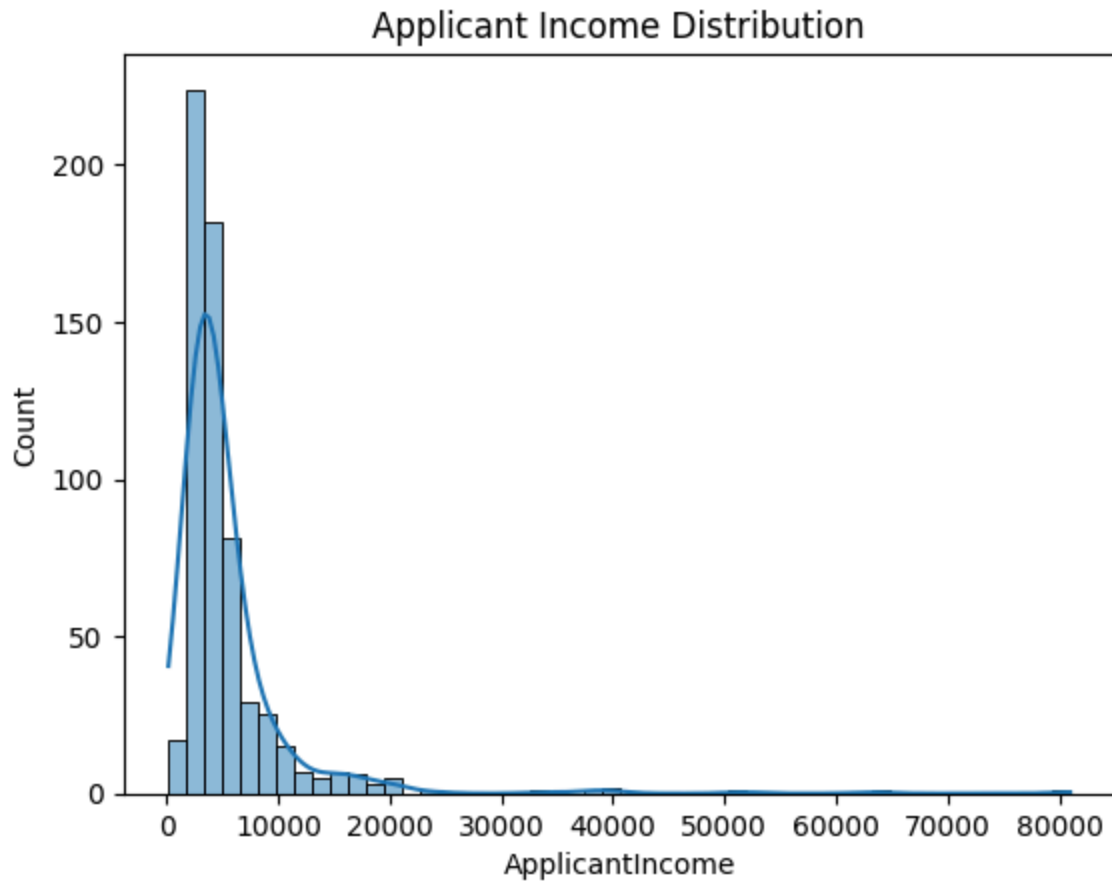


In [11]:

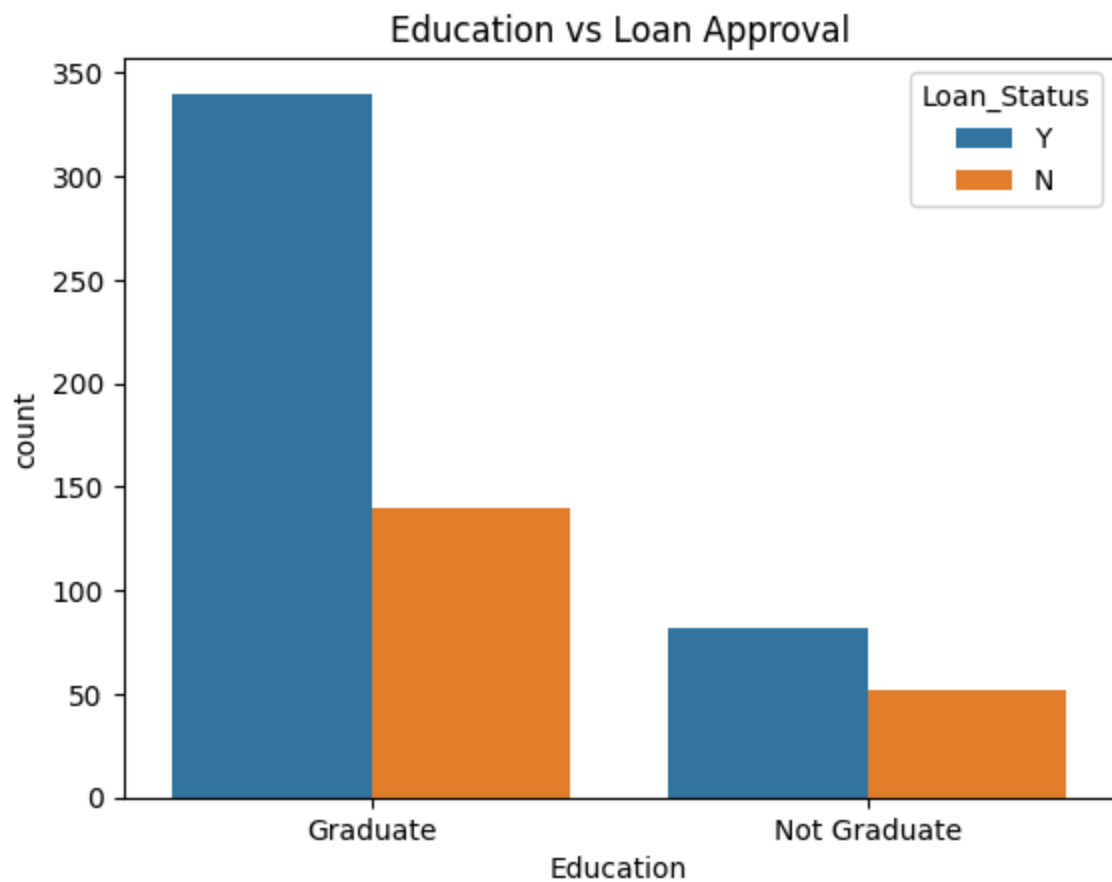
```
sns.histplot(df['ApplicantIncome'], kde=True)

plt.title("Applicant Income Distribution")
```

```
plt.show()
```



```
In [12]: sns.countplot(  
    x='Education',  
    hue='Loan_Status',  
    data=df  
)  
  
plt.title("Education vs Loan Approval")  
  
plt.show()
```



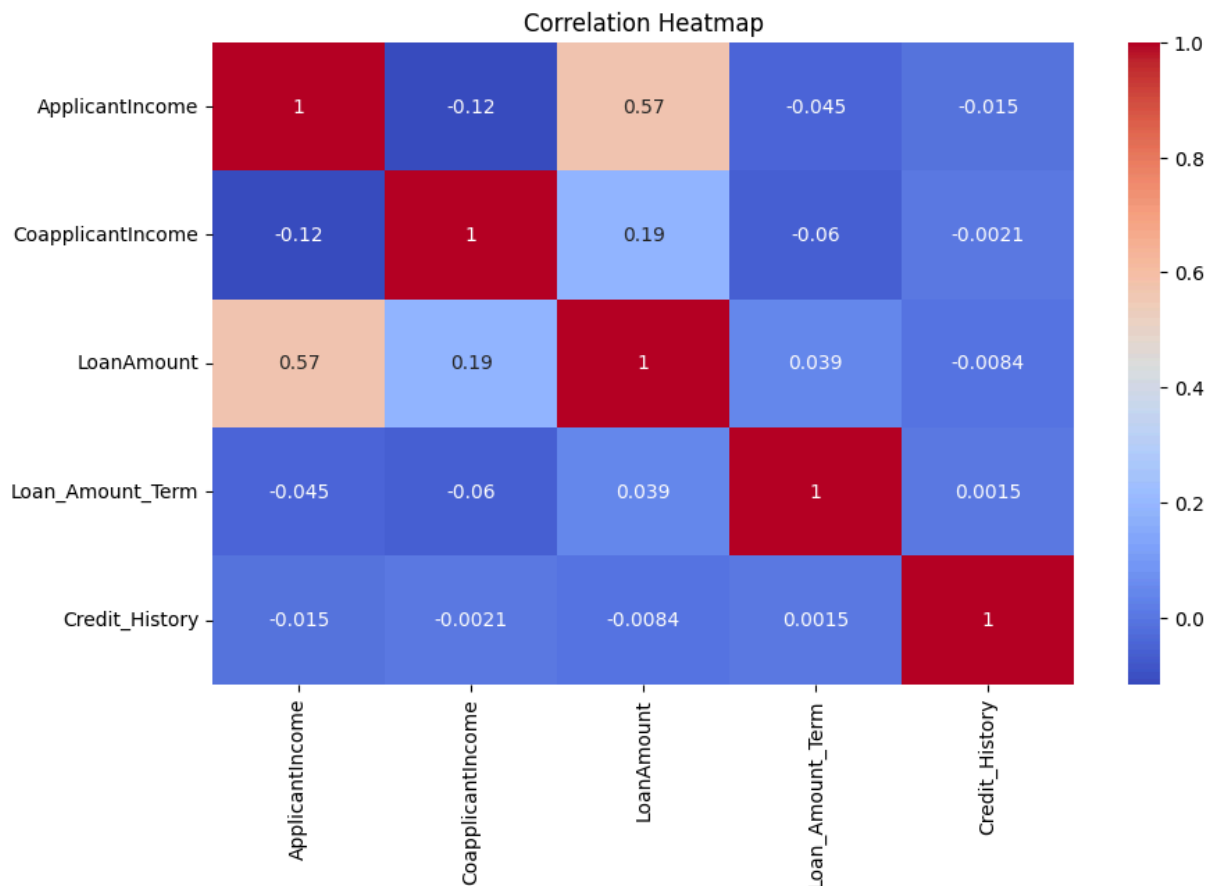
```
In [13]: numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(10,6))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap")

plt.show()
```



```
In [14]: X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']
```

```
In [15]: le = LabelEncoder()
y = le.fit_transform(y)
```

```
In [16]: num_cols = X.select_dtypes(include=np.number).columns
cat_cols = X.select_dtypes(exclude=np.number).columns
```

```
In [17]: numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

```
In [18]: categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),

    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])
```

```
In [19]: preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, num_cols),
```



```

        ('cat', categorical_transformer, cat_cols)
    ]
)

```

```

In [20]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

In [21]: X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

```

```

In [22]: smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train_processed,
    y_train
)

```

```

In [23]: lr_model = LogisticRegression()

lr_model.fit(
    X_train_smote,
    y_train_smote
)

```

```

Out[23]: ▾ LogisticRegression ⓘ ?
          ► Parameters

```

```

In [24]: lr_preds = lr_model.predict(X_test_processed)

lr_probs = lr_model.predict_proba(
    X_test_processed
)[: , 1]

```

```

In [25]: print("Precision:",
    precision_score(y_test, lr_preds))

print("Recall:",
    recall_score(y_test, lr_preds))

print("F1 Score:",
    f1_score(y_test, lr_preds))

print("ROC AUC:",
    roc_auc_score(y_test, lr_probs))

```

Precision: 0.8631578947368421  
 Recall: 0.9647058823529412  
 F1 Score: 0.9111111111111111  
 ROC AUC: 0.8687306501547988

```
In [26]: rf_model = RandomForestClassifier(
          random_state=42
        )

rf_model.fit(
    X_train_smote,
    y_train_smote
)
```

```
Out[26]: ▼ RandomForestClassifier ⓘ ?
          ► Parameters
```

```
In [27]: rf_preds = rf_model.predict(
          X_test_processed
        )

rf_probs = rf_model.predict_proba(
    X_test_processed
)[: , 1]
```

```
In [28]: print("Precision:",
              precision_score(y_test, rf_preds))

print("Recall:",
      recall_score(y_test, rf_preds))

print("F1 Score:",
      f1_score(y_test, rf_preds))

print("ROC AUC:",
      roc_auc_score(y_test, rf_probs))
```

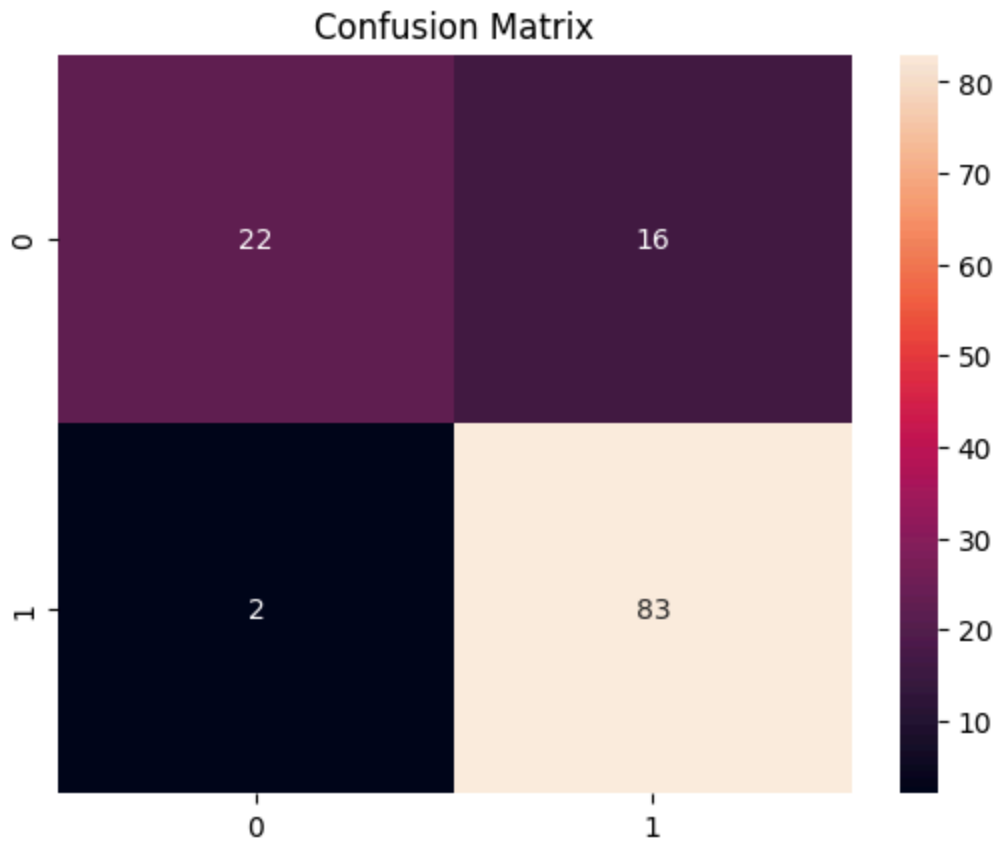
Precision: 0.8383838383838383  
 Recall: 0.9764705882352941  
 F1 Score: 0.9021739130434783  
 ROC AUC: 0.7846749226006192

```
In [29]: cm = confusion_matrix(y_test, rf_preds)

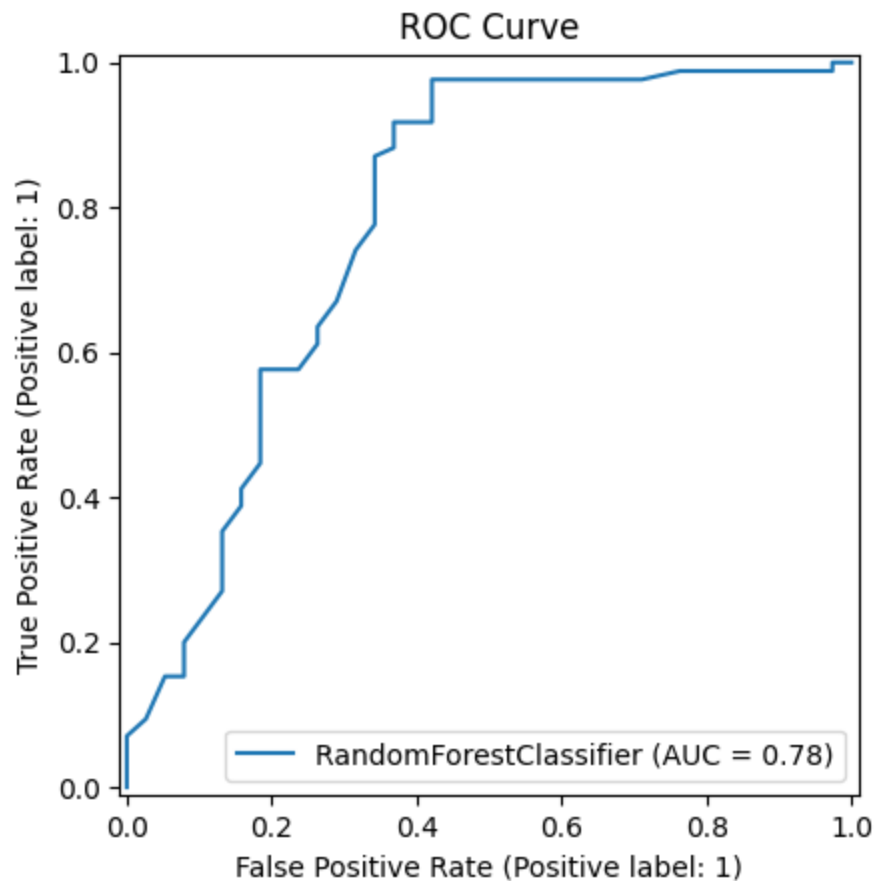
sns.heatmap(
    cm,
    annot=True,
    fmt='d'
)

plt.title("Confusion Matrix")

plt.show()
```



```
In [30]: RocCurveDisplay.from_estimator(  
    rf_model,  
    X_test_processed,  
    y_test  
)  
  
plt.title("ROC Curve")  
  
plt.show()
```



```
In [33]: feature_names = preprocessor.get_feature_names_out()

importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': rf_model.feature_importances_
})

importance_df = importance_df.sort_values(
    by='Importance',
    ascending=False
)

print(importance_df.head(10))
```

|     | Feature                      | Importance |
|-----|------------------------------|------------|
| 4   | num__Credit_History          | 0.134706   |
| 0   | num__ApplicantIncome         | 0.082233   |
| 2   | num__LoanAmount              | 0.073227   |
| 1   | num__CoapplicantIncome       | 0.052174   |
| 509 | cat__Property_Area_Semiurban | 0.028890   |
| 3   | num__Loan_Amount_Term        | 0.022521   |
| 508 | cat__Property_Area_Rural     | 0.021974   |
| 500 | cat__Dependents_0            | 0.017721   |
| 510 | cat__Property_Area_Urban     | 0.016634   |
| 501 | cat__Dependents_1            | 0.015417   |

## Business Interpretation

The machine learning model successfully predicted loan approval outcomes using applicant financial and demographic information.

Credit history was the most important factor influencing loan approval decisions. Applicants with good repayment history had significantly higher approval chances.

Income-related features also contributed strongly to approval probability, suggesting financially stable applicants are considered lower risk.

Among the tested models, Logistic Regression achieved better overall performance based on ROC-AUC and F1 score. Random Forest achieved higher recall, while Logistic Regression provided a better balance between identifying eligible applicants and reducing risky approvals.

The model achieved strong precision and ROC-AUC scores, indicating it can effectively reduce risky loan approvals while maintaining reliable applicant selection.

For deployment, a threshold of 0.6 is recommended to improve approval quality and reduce financial risk.

## Conclusion

This project developed a machine learning pipeline for predicting loan approval decisions.

The workflow included:

- Data preprocessing
- Exploratory Data Analysis
- Handling missing values
- Encoding categorical variables
- Feature scaling
- Class imbalance handling using SMOTE
- Model training and evaluation

Logistic Regression produced the best overall performance based on ROC-AUC and F1 score, making it the preferred model for this loan approval prediction task.

The project demonstrates how machine learning can support financial institutions in making faster and more reliable loan approval decisions.

In [ ]: